Validation des formulaires en SwiftUI

420-249-AH - Programmation mobile iOS

1. Pourquoi valider un formulaire ?

Un formulaire ne doit pas accepter automatiquement toutes les valeurs saisies. Avant d'enregistrer, l'application vérifie que les données respectent certaines règles :

- Un champ obligatoire ne doit pas être vide
- Un courriel doit avoir un format acceptable
- Une quantité doit respecter des limites
- Deux valeurs doivent être compatibles (date de fin après date de début)
- Une date doit être valide

L'objectif est d'éviter l'enregistrement de données incorrectes ou incomplètes, et de guider l'utilisateur vers la correction.

2. Une règle par champ

Chaque règle devient une propriété calculée de type Bool. formulaireValide les combine :

```
var nomValide: Bool {
    !nom
        .trimmingCharacters(in: .whitespacesAndNewlines)
        .isEmpty
}

var courrielValide: Bool {
    courriel.contains("@") &&
    courriel.contains(".")
}

var formulaireValide: Bool {
    nomValide &&
    courrielValide
}
```

- trimmingCharacters retire les espaces : une chaîne contenant seulement des espaces n'est pas un nom valide
- La vérification du courriel est volontairement simple : elle contrôle une structure minimale (@ et .), pas tous les formats possibles

3. Règles courantes

Règle	Expression
Longueur	`titre.count >= 3 && titre.count <= 50`
Mot de passe	`motDePasse.count >= 8`
Confirmation	`motDePasse == confirmation`
Plage numérique	`quantite >= 1 && quantite <= 20`
Date non passée	`dateReservation >= debutDeJournee`
Dates dépendantes	`dateFin >= dateDebut`
Champ facultatif	`telephone.isEmpty || telephone.count >= 10`
Règle conditionnelle	`!ajouterDate || dateSouhaitee >= debutDeJournee`

Pour accepter la date du jour, on compare avec le début de la journée : Calendar.current.startOfDay(for: Date()). Comparer avec Date() refuserait aujourd'hui dès que l'heure choisie est passée.

- Un champ facultatif est valide lorsqu'il est vide ; s'il est rempli, il doit respecter la règle
- Une règle conditionnelle ne s'applique que si l'option correspondante est activée
- Un Picker, un Stepper ou un DatePicker limite déjà les valeurs, mais cela ne remplace pas la validation du formulaire

4. Quand afficher les erreurs ?

Afficher toutes les erreurs dès l'ouverture du formulaire donne une mauvaise expérience. On attend une tentative d'enregistrement, ou on valide pendant la saisie seulement si le champ n'est pas vide :

```
// Après une tentative d'enregistrement
Button("Enregistrer") {
    tentativeEnregistrement = true
    if formulaireValide {
        enregistrer()
    }
}

if tentativeEnregistrement && !nomValide {
    Text("Le nom est obligatoire.")
        .font(.caption)
        .foregroundStyle(.red)
}

// Pendant la saisie : seulement si le champ n'est pas vide
if !courriel.isEmpty && !courrielValide {
    Text("Adresse courriel invalide.")
        .font(.caption)
        .foregroundStyle(.red)
}
```

5. Le bouton Enregistrer

Stratégie	Quand l'utiliser
`.disabled(!formulaireValide)`	Formulaire simple : l'utilisateur ne peut pas appuyer tant que les données sont invalides
Bouton actif et erreurs affichées	Formulaire plus complexe : appuyer fait apparaître les erreurs, ce qui est plus clair

6. Retourner un message d'erreur

Un Bool convient à une règle simple. Lorsque plusieurs erreurs sont possibles pour un champ, un String? retourne le message à afficher, ou nil si la valeur est valide :

```
var erreurMotDePasse: String? {
    if motDePasse.isEmpty {
        return "Le mot de passe est obligatoire."
    }
    if motDePasse.count < 8 {
        return "Minimum 8 caractères."
    }
    return nil
}

// Dans le body
if let erreur = erreurMotDePasse {
    Text(erreur)
        .font(.caption)
        .foregroundStyle(.red)
}
```

7. Valider avant d'enregistrer

Règle importante : on valide d'abord, puis on modifie ou on enregistre les données, jamais l'inverse. guard arrête l'action dès qu'une donnée est invalide :

```
func valider() {
    tentativeEnregistrement = true

    guard nomValide else {
        champActif = .nom
        return
    }
    guard courrielValide else {
        champActif = .courriel
        return
    }
    enregistrer()
}
```

- @FocusState place le curseur sur le premier champ invalide (vu avec les formulaires)
- Le message précis s'affiche près du champ ; pour une erreur générale, on peut ajouter une alerte (vue avec les feuilles)
- Avec un ViewModel (vu avec MVVM), ces propriétés calculées s'y placent : la vue se limite à afficher les erreurs
- La validation vérifie si les données sont acceptables ; les conserver (persistance) est une autre responsabilité

8. Exemple Formulaire de contact

Première partie : les états et les règles de validation :

```
struct ContactFormView: View {
    enum Champ {
        case nom
        case courriel
    }

    @State private var nom = ""
    @State private var courriel = ""
    @State private var tentative = false
    @FocusState private var champActif: Champ?

    var erreurNom: String? {
        let texte = nom.trimmingCharacters(in: .whitespacesAndNewlines)
        return texte.isEmpty ? "Le nom est obligatoire." : nil
    }

    var erreurCourriel: String? {
        if courriel.isEmpty {
            return "Le courriel est obligatoire."
        }
        if !courriel.contains("@") || !courriel.contains(".") {
            return "Le courriel est invalide."
        }
        return nil
    }

    // ...
}
```

Deuxième partie : l'interface, l'affichage des erreurs et la validation :

```
struct ContactFormView: View {
    // États et règles ci-dessus

    var body: some View {
        Form {
            Section("Informations") {
                TextField("Nom", text: $nom)
                    .focused($champActif, equals: .nom)
                afficher(erreurNom)

                TextField("Courriel", text: $courriel)
                    .focused($champActif, equals: .courriel)
                afficher(erreurCourriel)
            }

            Button("Enregistrer") {
                valider()
            }
        }
    }

    @ViewBuilder
    private func afficher(_ message: String?) -> some View {
        if tentative, let texte = message {
            Text(texte)
                .font(.caption)
                .foregroundStyle(.red)
        }
    }

    private func valider() {
        tentative = true

        guard erreurNom == nil else {
            champActif = .nom
            return
        }
        guard erreurCourriel == nil else {
            champActif = .courriel
            return
        }
        print("Enregistrement")
    }
}
```

9. Messages d'erreur

Un message d'erreur doit être court, précis, placé près du problème et compréhensible :

À préférer	À éviter
Le nom est obligatoire.	Erreur de validation.
Minimum 8 caractères.	Valeur incorrecte
Le courriel est invalide.	Erreur 12

10. À retenir

Notion	À retenir
`Bool`	Indique si une valeur est valide
`String?`	Retourne un message d'erreur, ou nil
`Propriétés calculées`	nomValide, courrielValide, formulaireValide
`Moment`	Après une tentative d'enregistrement, ou pendant la saisie
`guard`	Arrête l'enregistrement si les données sont invalides

Une bonne validation permet de détecter les données incorrectes, d'expliquer le problème, d'empêcher l'enregistrement et de guider l'utilisateur vers la correction.

11. Exercice Validation d'une réservation

Créer un formulaire de réservation contenant Nom, Courriel, Nombre de personnes, Date de réservation et Notes.

1. Règles
Ajouter les règles suivantes, avec un message sous chaque champ invalide :

- Nom obligatoire
- Courriel valide
- Nombre de personnes entre 1 et 8
- Date non passée (la date du jour est acceptée)

2. Tentative d'enregistrement
Les erreurs ne doivent apparaître qu'après une tentative d'enregistrement (variable tentativeEnregistrement).

3. Bouton
Créer formulaireValide. Le bouton Enregistrer ne doit effectuer aucune action si le formulaire est invalide.

12. Exercice Premier champ invalide

Utiliser @FocusState pour placer automatiquement le curseur sur le premier champ invalide lorsque l'utilisateur appuie sur Enregistrer.